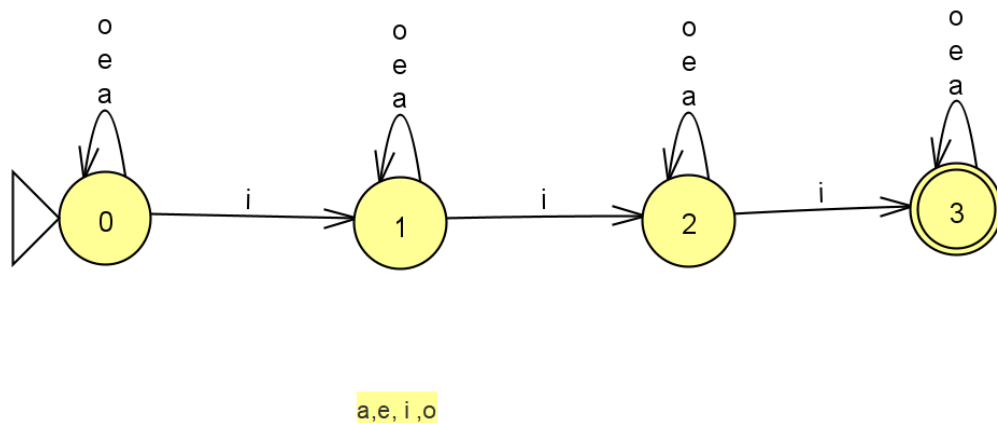


The language defined by an automaton is a formal language. Examples of formal languages include programming languages or JSON (Javascript Object Notation) etc. The words belonging to a formal language can be defined by an automaton. A DFA tells us whether a word (input) belongs to the language or not.

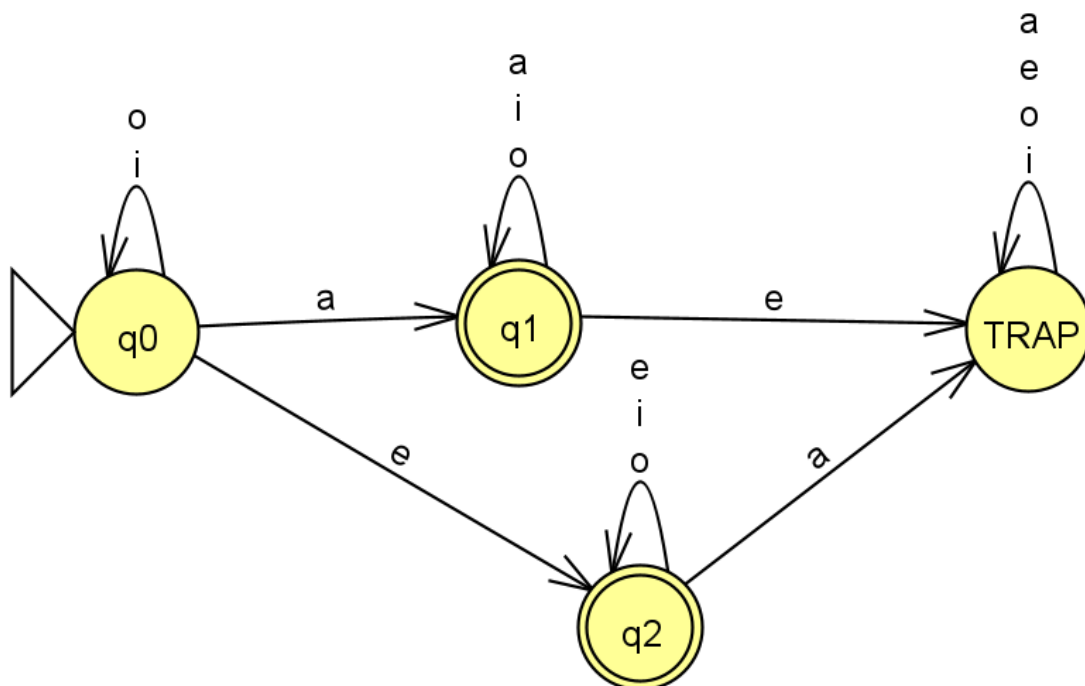
Spoken languages are usually not formal, for example, they have many exceptions.

The automaton M defines the language L : $L(M) = \{x: \delta(s, x) \in F\}$, where s is the initial state of the automaton and $x = (x_1x_2...x_n)$, $x_i \in \Sigma$, in other words, starting from the initial state s with input x , you end up in a final state when the input is completely consumed.

A DFA is not very impressive in counting and it remembers nothing. Computation easily produces many states. Let's make an example where $\Sigma = \{a, e, i, o\}$ and the word must contain (at least) three i -characters.

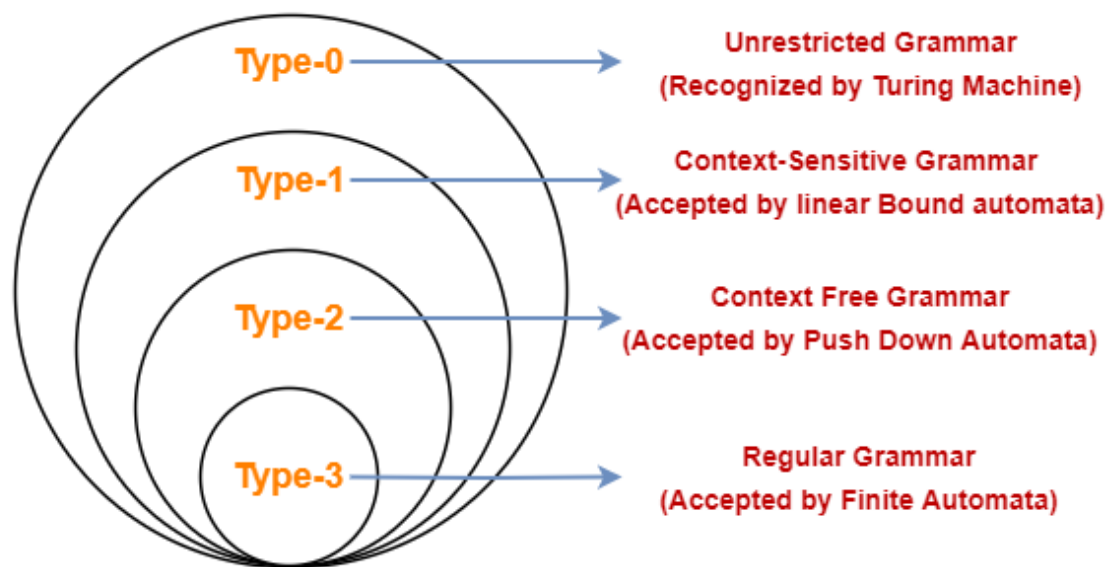


Example 2, the same alphabet and the word must contain either an a or an e, one or more, but not both.



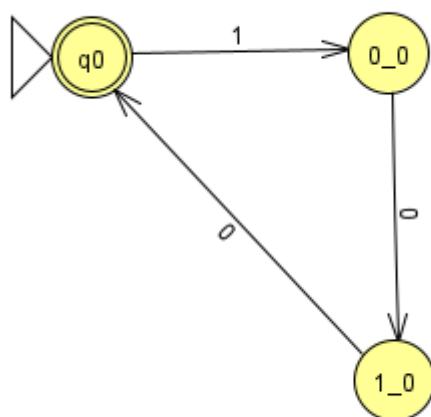
How would you make an automaton for a language $\Sigma=\{a,b,c\}$, $L(M)=\{x, |a|=|b|\}$ i.e., words that have as many a letters as there are b letters?

Languages and automata



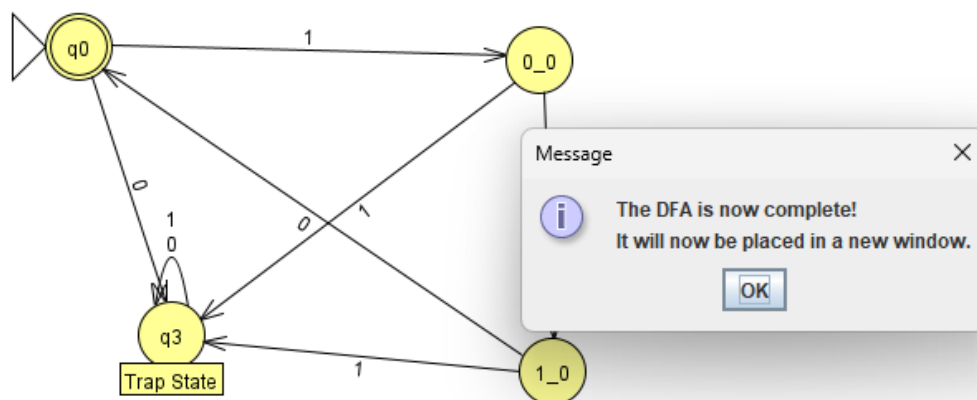
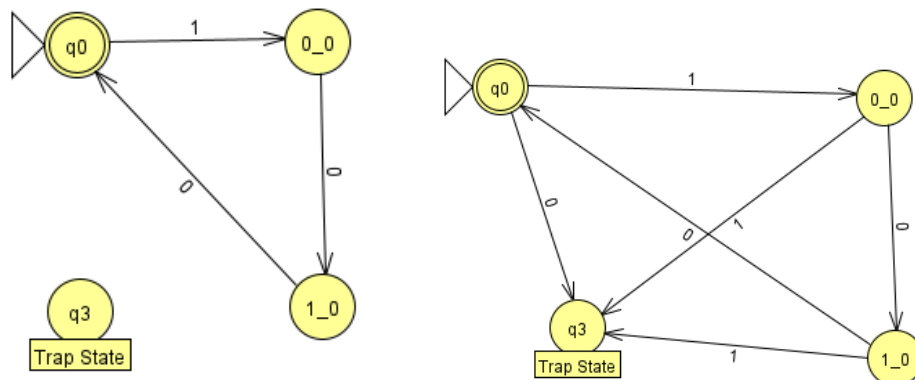
DFA Trap state

If a DFA is not complete, i.e., there are no transitions for all alphabet letters from every state, a "trap state" can be added."



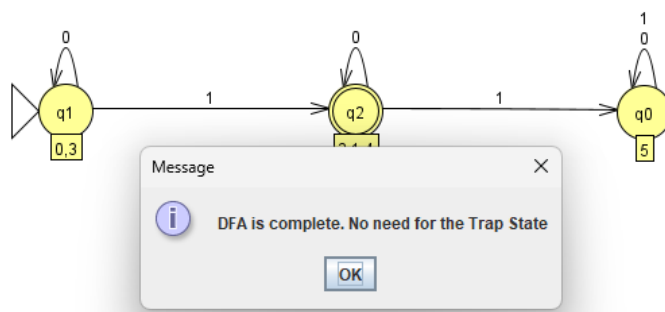
This automaton is not complete, for example, from state **1_0** there is no transition on **1**.

Convert->Add Trap State



JFLAP does not necessarily require a trap state because it assumes it automatically.

Note! JFLAP does not allow creating a trap state for an automaton that is complete:



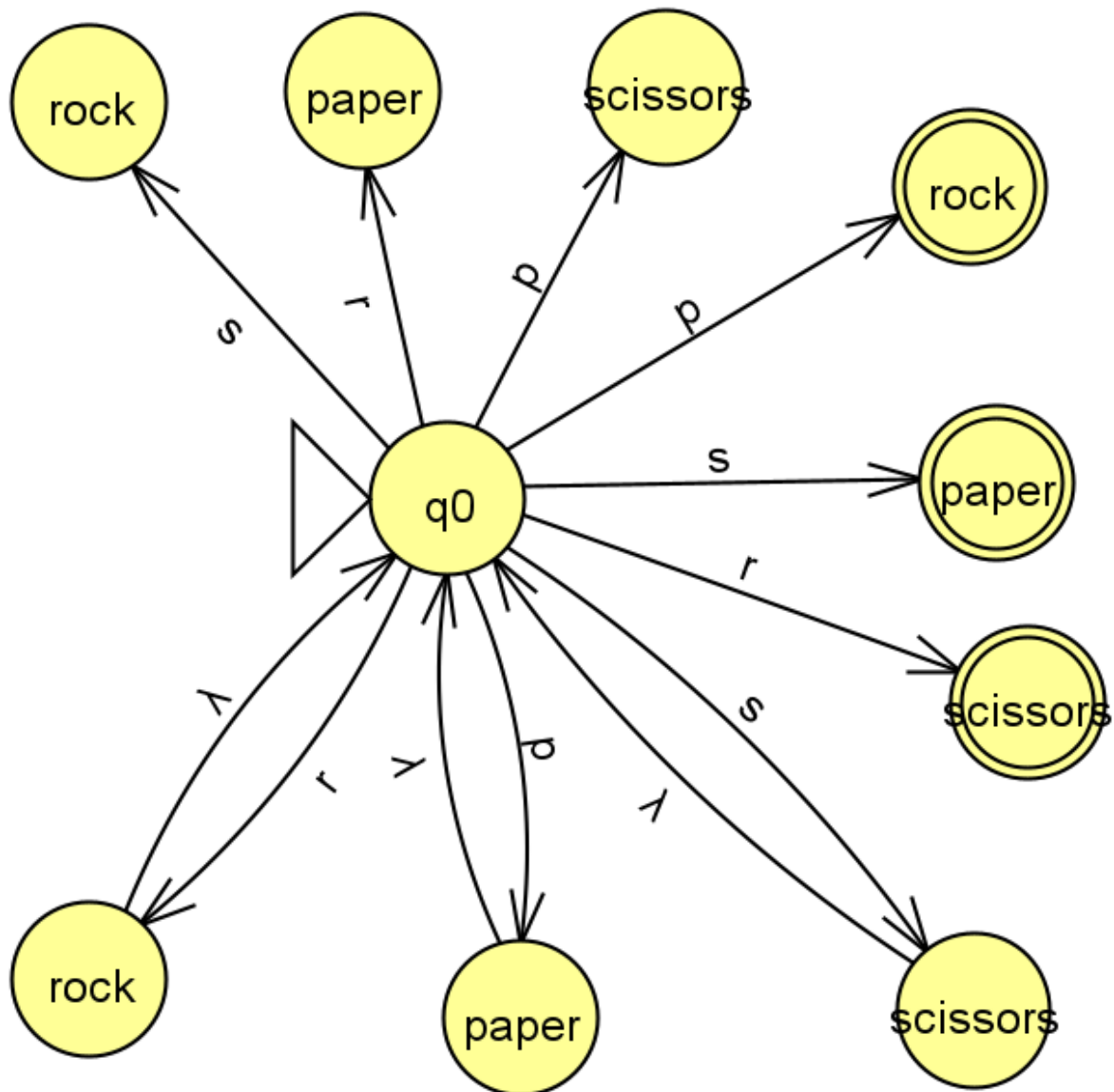
NFA

Non-deterministic FA (NFA) is an automaton in which from a certain state with a certain input symbol there can be multiple transition options. The option is chosen randomly (non-determinism).

The deterministic transition function of a DFA $\delta = Q \times \Sigma \rightarrow Q$ changes to $\delta = Q \times \Sigma \rightarrow P(Q)$, where $P(Q)$ is the set of all subsets of Q . That is, when a state-input pair in a DFA maps to one state, in an NFA it maps to a subset of states.

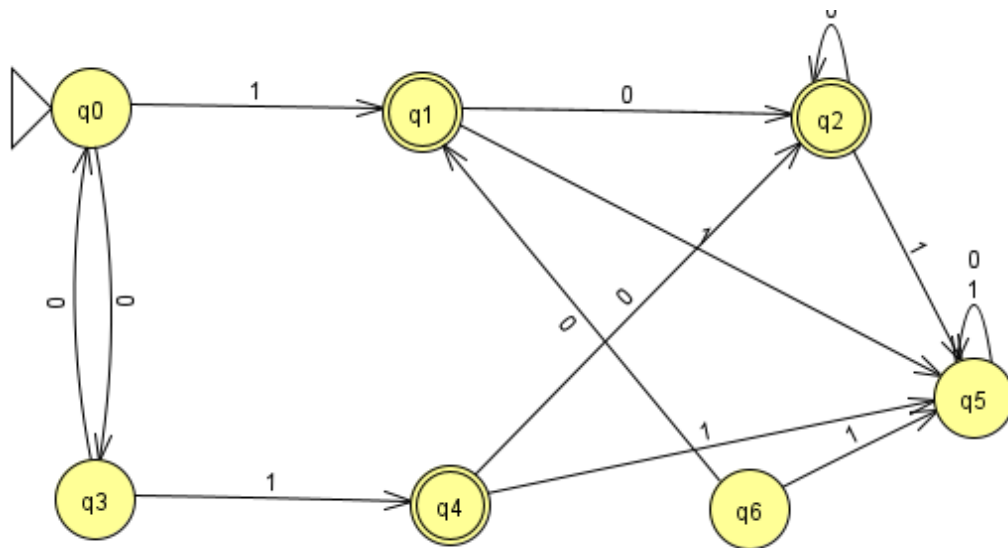
P = Power Set. As a reminder, Power Set – Wikipedia

Lecture example, RPS (rock, paper, scissors)



DFA transition table

The transition table describes the transitions of the DFA



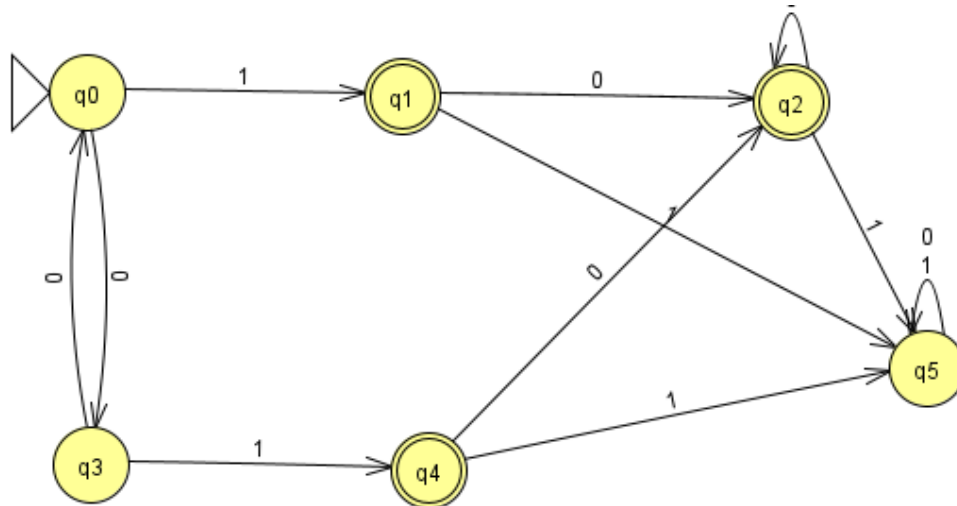
According to the example above:

	1	0
→ q0	q1	q3
q1	q5	q2
q2	q5	q2
q3	q4	q0
q4	q5	q2
q5	q5	q5
q6	q5	q1

Minimization algorithm

A DFA can be minimized. Minimization removes unnecessary transitions and states. The minimized automaton accepts the same language as the original.

1. Remove unreachable states, here q6



2. Divide the states into final states and initial states

P_0 (Partition 0) = $\{\{q1, q2, q4\} \{q0, q3, q5\}\}$

These partitions eventually form the new states.

After this, the states of both sets are examined in pairs, resulting in the next partitioning P_1

$\{q1, q2, q4\}$

q1 q2	1	0
q1	q5	q2
q2	q5	q2

These are the same, no action is taken.

q1 q4	1	0
q1	q5	q2
q4	q5	q2

These are the same, no action is taken.

q2 q4	1	0
q2	q5	q2
q4	q5	q2

These are also the same, and of course they must be, because the previous ones were

{q0,q3,q5}

q0 q3	1	0
q0	q1	q3
q3	q4	q0

Transitions with 0 are in the same partition $p_0 \{q_0, q_3, q_5\}$ and transitions with 1 are also in the same partition $\{q_1, q_2, q_4\}$, so these do not differ from each other.

q0 q5	1	0
q0	q1	q3
q5	q5	q5

Now transitions with 1 lead to different partitions, so q_5 differs and its own partition $\{q_5\}$ must be made

P1: $\{\{q_1, q_2, q_4\}, \{q_0, q_3\}, \{q_5\}\}$

The process is repeated until the sets no longer change. $\{q_5\}$ no longer needs to be examined because it is already a single state.

{q1,q2,q4}

q1 q2	1	0
q1	q5	q2
q2	q5	q2

These remain unchanged, even though q_5 is in a different set than q_2 , they are reached by the same transitions

q1 q4	1	0
q1	q5	q2
q4	q5	q2

These are also the same, no actions are required.

q2 q4	1	0
q2	q5	q2
q4	q5	q2

As are these

-> **{q1,q2,q4} does not "divide "**

{q0,q3}

q0 q3	1	0
q0	q1	q3
q3	q4	q0

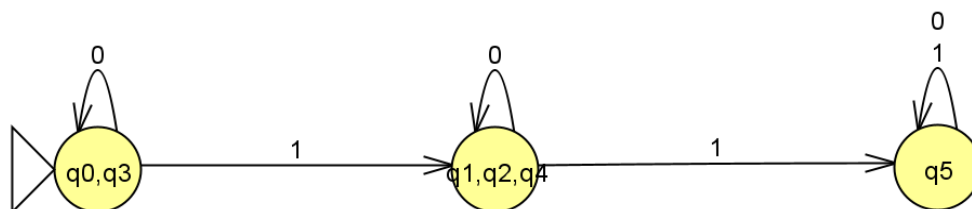
Transitions with 0 are in the same partition of P0 {q0,q3} and transitions with 1 are also in the same partition {q1,q2,q4}, so these do not differ from each other, and this also no longer "splits".

In the end, the states formed are **{q1,q2,q4}{q0,q3}{q5}**

And the transitions are:

	0	1
{q1,q2,q4}	{q1,q2,q4}	{q5}
{q0,q3}	{q0,q3}	{q1,q2,q4}
{q5}	{q5}	{q5}

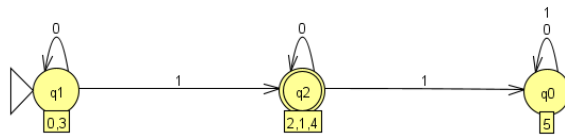
Since {q1,q2,q4} was the initial state, the automaton looks like this.



Is this minimal???

As a regular expression 0^*10^*

JFLAP can also do this automatically: Convert->MinimizedDFA



Input	Result
100	Accept
1100	Reject
100110	Reject
00	Reject
01	Accept
10	Accept
11	Reject
000000001	Accept
0000100000010	Reject
1	Accept
010	Accept
110011	Reject

Let's also have JFLAP compare these with each other:

